Лабораторна робота 2

Організація введення/виведення цифрових сигналів у мікроконтролер

Мета: ознайомитися з принципами введення/виведення цифрових сигналів. Вивчити роботу дискретних електронних компонентів (світлодіод, пьезовипромінювач, кнопка тощо) та використання плати Arduino. Ознайомитись з середовищем розробки Arduino IDE.

Завдання роботи:

Завдання 1: Ввести програму на рисунку 2.17. Переконатися в її працездатності. В якості кнопки використати кнопку S1 макету.

Завдання 2: Ввести програму на рисунку 2.18. Переконатися в її працездатності. Змінити швидкість миготіння.

Завдання 3: Використовуючи світлодіод (13 вивід) на платі, реалізувати формування сигналу "SOS" з використанням азбуки Морзе. За одиницю часу приймати тривалість однієї точки. Тривалість тире дорівнює трьом точкам. Пауза між елементами одного знака – одна точка, між знаками в слові – 3 точки, між словами – 7 точок.

Завдання 4: Повторити завдання 3, використавши пьезовипромінювач.

Завдання 5: Реалізувати функцію “Мерехтливі вогні” у відповідності варіанту (див. таблицю 5.1).

Варіант	Стан1/Затримка, мс	Стан2/Затримка, мс	Стан3/Затримка, мс	Стан4/Затримка, мс
6	0, 0, 0, 1/0.7	0, 0, 1, 1/0.7	0, 1, 1, 1/0.7	1, 1, 1, 1/0.7

Завдання 6: Використавши функцію, що реалізована у завданні 5, створити функцію “Дуальні мерехтливі вогні”, яка змінює направлення мерехтіння при натисканні другої кнопки S2.

Завдання 7: Ввести програму на рисунку 2.19. Переконатися в її працездатності.

Завдання 8: Використовуючи датчик вібрацій (підключити до виводу 9) і світлодіод на налагоджувальної платі, реалізувати систему оповіщення, використовуючи світлові спалахи (частота 0,5 Гц) і звуковий сигнал з

частотою випромінювання 500 Гц. Також формувати повідомлення в послідовний монітор.

Хід роботи:

Завдання 1:

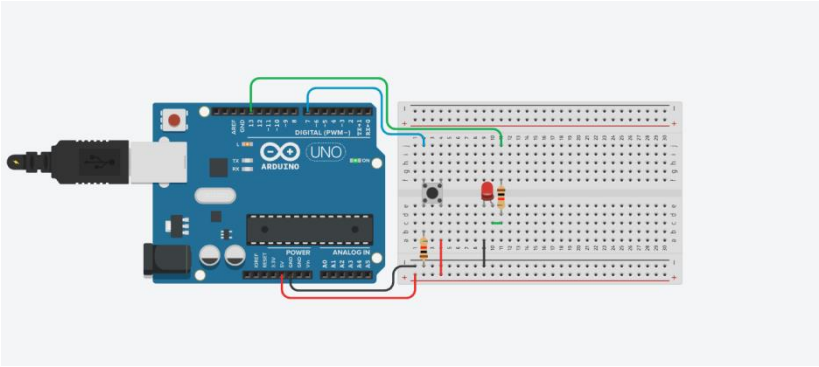


Рисунок 2.1 - програма з вмиканням лед лампочки через кнопку

Програма: https://www.tinkercad.com/things/bezwIPyuOVV-lab21

Загальна ідея:

Програма постійно читає сигнал на піні 7 і одразу ж видає те саме значення на пін 13, до якого підключений світлодіод. Є сигнал на вході, і світлодіод горить. Немає сигналу, і він вимкнений

Як працює:

Оголошення змінних

int ledPin = 13;

int inPin = 7;

bool val = 0;

l**edPin = 13**: номер піна зі світлодіодом

inPin = 7: номер піна, з якого читається вхідний сигнал кнопки

val = 0: змінна типу **bool**, у ній зберігається поточний стан входу: **0** (**LOW**) або **1** (**HIGH**)

Функція **setup()**

Виконується один раз при старті або перезавантаженні плати

pinMode(ledPin, OUTPUT);

pinMode(inPin, INPUT);

Пін **13** налаштовується як вихід, тобто **Arduino** сама подає на нього напругу

Пін **7** налаштовується як вхід, тобто **Arduino** вимірює напругу, яка на нього приходить

Функція **loop()**
Виконується безперервно, знову і знову
val = digitalRead(inPin);
digitalWrite(ledPin, val);
digitalRead(inPin)
зчитується стан кнопки й одразу виводиться на світлодіод. Кнопка натиснута дає **HIG**H, і світлодіод горить. Кнопка відпущена дає **LOW**, і світлодіод вимкнений. У схемі це забезпечує резистор **10 кОм**, який підтягує пін **7** до **GND**.

Режим **INPUT** означає, що пін «висить у повітрі», якщо до нього нічого не підключено або кнопка розімкнена. Він ловить наводки, і світлодіод може хаотично блимати.
Увімкнути внутрішню підтяжку до живлення:
pinMode(inPin, INPUT_PULLUP);

Тоді кнопку підключають між піном **7 і GND**. У такому випадку логіка інвертована: натиснуто дає **LOW**, відпущено дає **HIGH**. Щоб світлодіод горів при натисканні, треба писати **digitalWrite(ledPin, !val)**;
Завдання 2:

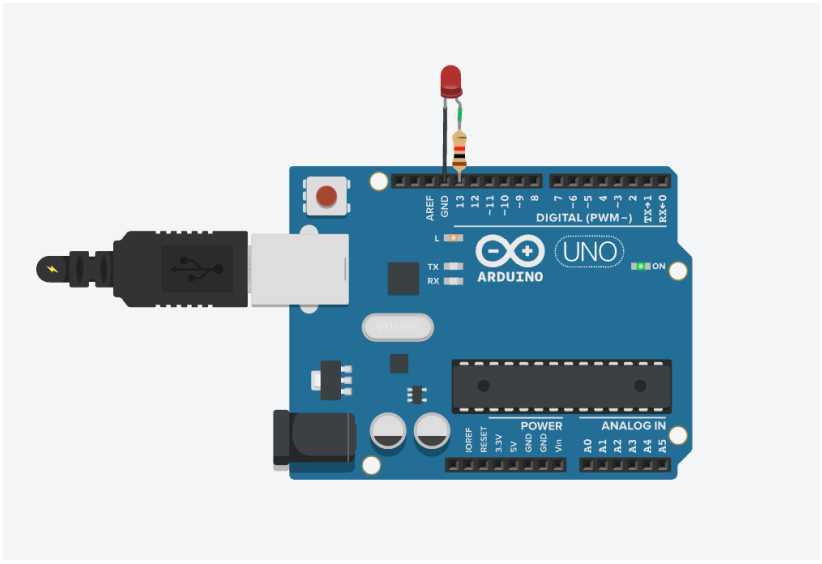


Рисунок 2.2 - програма з миготінням лед лампи кожні 0.5с
Програма: https://www.tinkercad.com/things/cyTEB7qfvlE-lab22

Як працює програма:
int led = 13;

Оголошується змінна типу **int** (ціле число) з іменем **led** і значенням 13. Це номер цифрового піна, до якого підключений світлодіод

```
void setup() {
    pinMode(led, OUTPUT);
}
```

setup() виконується один раз після подачі живлення або натискання **Reset**. Функція **pinMode**(**пін, режим**) задає, як працюватиме пін. **OUTPUT** означає, що пін є виходом: мікроконтролер сам подаватиме на нього напругу (**5 В**) або ставитиме **0 В**. Без цього рядка d**igitalWrite** працював би некоректно (світлодіод ледве світився б через внутрішню підтяжку)

```
void loop() {
    digitalWrite(led, HIGH); /* встановлення високого логічного рівня */
    delay(500);              /* змінено затримку з 1 с до 0.5 с */
    digitalWrite(led, LOW);  /* встановлення низького логічного рівня */
    delay(500);              /* змінено затримку з 1 с до 0.5 с */
}
```

loop() виконується безперервно: після останнього рядка програма повертається на перший. Що робить кожен рядок:

digitalWrite(led, HIGH): на піні **13** з'являється **5 В**. Струм тече через світлодіод, і він горить

delay(500): програма зупиняється на **500** мілісекунд (**0,5 с**). Увесь цей час світлодіод залишається у стані **HIGH**

digitalWrite(led, LOW): на піні **0 В**. Струму немає, світлодіод не горить

delay(500): ще 0,5 с у вимкненому стані

Завдання 3:

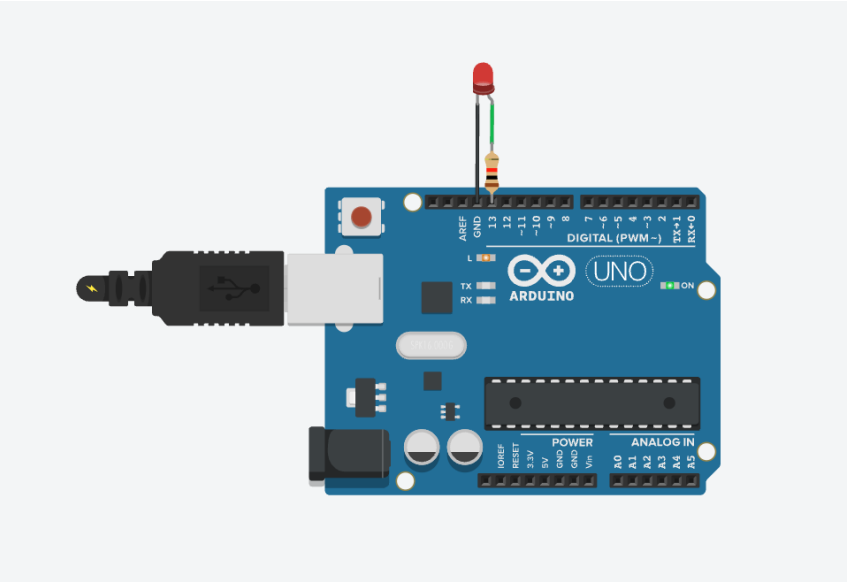


Рисунок 2.3 - програма формує сигнал "SOS"

Програма: https://www.tinkercad.com/things/1nZDbwx1wLA-lab23?sharecode=WSxFbTdbquXIPWq9EamOizXF9gttZ6BM-YhtGHA98a4

Як працює програма:

Світлодіод на піні **13** безкінечно повторює сигнал бідування **SOS** азбукою Морзе: три коротких спалахи, три довгих, знову три коротких, а потім довга пауза.

Початок програми

int ledPin = 13;

Змінна зберігає номер піна, до якого підключений світлодіод

void setup()

{

pinMode(ledPin, OUTPUT);

}

Виконується один раз при старті. Пін 13 налаштовується як вихід, тобто Arduino сама керує напругою на ньому

HIGH + delay(200) крапка 200 мс (1 точка)

HIGH + delay(600) тире 600 мс (3 точки)

LOW + delay(200) пауза між елементами однієї літери 200 мс (1 точка)

LOW + delay(400) пауза між літерами 400 мс (2 точки)

LOW + delay(1200) пауза між словами 1200 мс (6 точок)

Літера **S**: три крапки

// S = ...

digitalWrite(ledPin, HIGH); // світлодіод горить
delay(200); // 200 мс
digitalWrite(ledPin, LOW); // світлодіод гасне
delay(200); // пауза 200 мс
digitalWrite(ledPin, HIGH); // друга крапка
delay(200);
d**igitalWrite(ledPin, LOW);**
delay(200);
digitalWrite(ledPin, HIGH); // третя крапка
delay(200);
digitalWrite(ledPin, LOW); // гасне, пауза після літери буде нижче

Кожна крапка це «горить **200 мс**, не горить **200 мс**». Після третьої крапки немає **delay(200**): одразу йде пауза між літерами. Тривалість літери **S**: **200 + 200 + 200 + 200 + 200 = 1000 мс**.

Пауза між літерами

// Пауза між S і O
delay(400);

Світлодіод уже вимкнений **(LOW)**, програма просто чекає **400 мс**.

Літера **O**: три тире

// O = ---
digitalWrite(ledPin, HIGH);
delay(600); // тире: горить 600 мс
digitalWrite(ledPin, LOW);
delay(200); // пауза між елементами
digitalWrite(ledPin, HIGH);
delay(600); // друге тире
digitalWrite(ledPin, LOW);
delay(200);
digitalWrite(ledPin, HIGH);
delay(600); // третє тире
digitalWrite(ledPin, LOW);

Структура та сама, що й у **S**, але світлодіод горить утричі довше. Тривалість літери **O**: **600 + 200 + 600 + 200 + 600 = 2200 мс**.

// Пауза між O і S
delay(400);

Друга літера **S** і завершення

Другий блок **S** ідентичний першому (**1000 мс**). Далі:

// Пауза перед повторенням SOS
delay(1200);

Це пауза між словами. Коли вона завершується, **loop()** починається спочатку, і **SOS** повторюється.

Завдання 4:

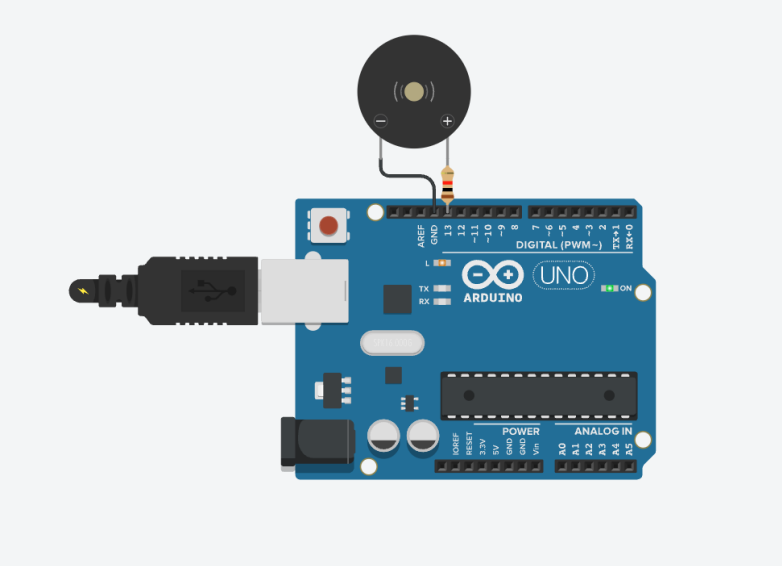


Рисунок 2.4 - програма формує сигнал "SOS" через пьезовипромінювач

Програма: https://www.tinkercad.com/things/jkiiM8jz3YH-lab24?sharecode=SG-GO5l6tTgmdTng57Qmk5aDNZr8s6-QI4rE3OgaGEU

Та сама послідовність SOS, але замість світла звучить звуковий сигнал: короткі звуки для крапок і довгі для тире.

Завдання 5:

Варіант	Стан1/Затримка, мс	Стан2/Затримка, мс	Стан3/Затримка, мс	Стан4/Затримка, мс
6	0, 0, 0, 1/0.7	0, 0, 1, 1/0.7	0, 1, 1, 1/0.7	1, 1, 1, 1/0.7

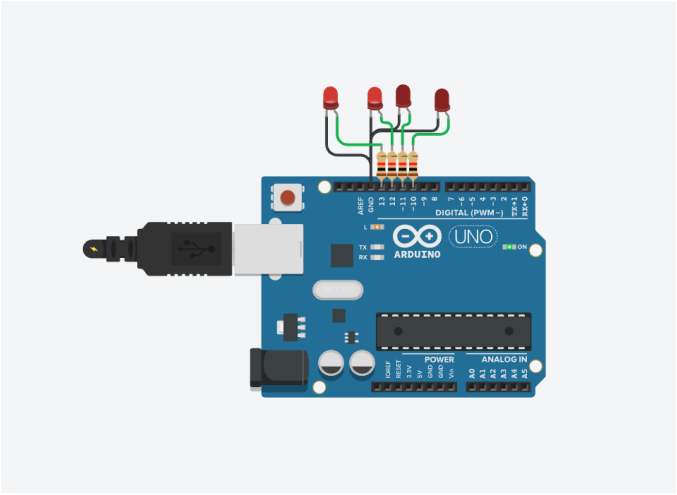


Рисунок 2.5 - в програмі реалізувана функція “Мерехтливі вогні”

Програма: https://www.tinkercad.com/things/61bzeNAoPzJ-lab25?sharecode=b9u49cdOwFBwihnhM5DtbQhrifWhhqdlbCm2Uh2AAcY

Як працює програма:

Змінні

int led1 = 13;

int led2 = 12;

int led3 = 11;

int led4 = 10;

Чотири світлодіоди

setup() налаштовує всі чотири піни як виходи.

loop(): чотири стани

Кожен стан записується записом із чотирьох цифр у порядку l**ed1, led2, led3, led4**. **1** означає, що світлодіод горить.

Стан	l**ed1 (13)**	**led2 (12)**	**led3 (11)**	**led4 (10)**	Затримка
1	**1**	**0**	**0**	**0**	700 мс
2	**1**	**1**	**0**	**0**	600 мс
3	**0**	**1**	**1**	**0**	500 мс
4	**0**	**0**	**1**	**1**	400 мс

У кожному стані код виставляє значення для всіх чотирьох пінів, а потім чекає **delay(...)**. Після четвертого стану **loop()** починається знову зі стану **1**

Горіння «зміщується» від **led1** до **led4**: один світлодіод, потім два, потім дві сусідні пари, які рухаються вбік. Затримки зменшуються від **700** до **400** мс, тому ефект поступово прискорюється. Повний цикл триває **700 + 600 + 500 + 400 = 2200** мс. Після стану **4** відбувається різкий стрибок назад до стану **1 (0011 → 1000)**.

Завдання 6:

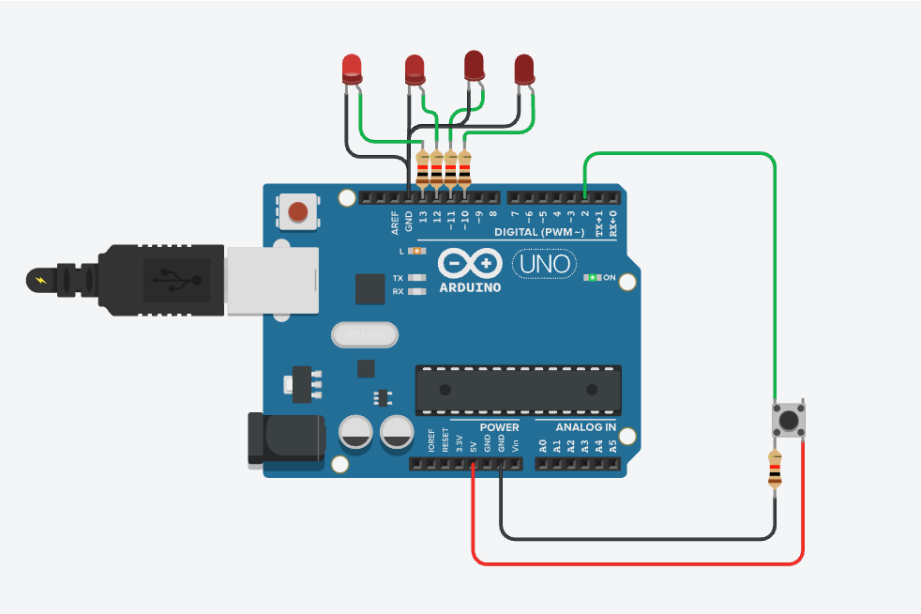


Рисунок 2.6 - в програмі реалізувана функція “Дуальні мерехтливі вогні”

Програма: https://www.tinkercad.com/things/irE3wbKsHL8-lab26?sharecode=R5ulvthxRP6zGCxkuBQJ82wpjlQlSEntQ83F3dvFsdg

Як працює програма:

```
const int led[4] = {10, 11, 12, 13};
const int S2 = 2;
```

led[4] це масив із чотирьох номерів пінів. Звернення **led[0**] дає **10**, **led[3]** дає **13**. Слово **const** означає, що значення не можна змінювати під час роботи. **S2** це пін кнопки

```
const bool states[4][4] = {
  {1, 0, 0, 0},
  {1, 1, 0, 0},
  {0, 1, 1, 0},
  {0, 0, 1, 1}
};
```

Це двовимірний масив: рядок це номер стану, стовпець це номер світлодіода. Наприклад, **states[2][1]** це «стан **3**, другий світлодіод», тобто **1**. Це та сама таблиця, що в завданя 2.5, тільки записана компактно

const int delays[4] = {700, 600, 500, 400};

Тривалість кожного стану в мілісекундах. **delays[i]** відповідає **states[i]**.

```
int idx = 0;
int dir = 1;
bool buttonOld = HIGH;
```

idx це номер поточного стану, від **0** до **3** (у масивах нумерація починається з нуля)

dir це напрямок: **+1** вперед, **-1** назад.

buttonOld це стан кнопки в попередній перевірці (**HIGH** означає не натиснута)

```
void setLeds(int i)
{
  for (int k = 0; k < 4; k++)
    digitalWrite(led[k], states[i][k]);
}
```

Цикл проходить усі чотири світлодіоди (**k = 0…3**) і для кожного записує на пін значення з таблиці для стану i. Так чотири рядки **digitalWrite** на кожен стан замінено одним циклом

```
void checkS2()
{
  bool button = digitalRead(S2);

  if (buttonOld == HIGH && button == LOW)
  {
     delay(30);
          if (digitalRead(S2) == LOW)
       dir = -dir;
  }
  buttonOld = button;
}
```

Функція шукає момент натискання кнопки та змінює напрямок.

Зчитується поточний стан кнопки. Пін **2** має внутрішню підтяжку, тому відпущена кнопка дає **HIGH**, а натиснута дає **LOW**.

Умова **buttonOld == HIGH && button == LOW** виявляє перехід з «відпущена» в «натиснута», тобто саме натискання. Утримування кнопки не дає повторних спрацювань.

delay(30) це антидребезг. Механічні контакти при замиканні «дзвенять» кілька мілісекунд, сигнал стрибає між **HIGH і LOW**. Затримка **30 мс** пропускає цей хаос.

Повторна перевірка **digitalRead(S2) == LOW** підтверджує, що кнопка справді натиснута, а не було випадкового імпульсу. Лише тоді **dir = -dir** розвертає напрямок.
buttonOld = button запам'ятовує стан для наступної перевірки.

```
void showState(int i)
{
  setLeds(i);

  int elapsed = 0;
  while (elapsed < delays[i])
  {
    checkS2();
    delay(10);
    elapsed += 10;
  }
}
```

Функція показує один стан і чекає потрібний час, не пропускаючи натискань.
setLeds(i) вмикає потрібну комбінацію.
elapsed рахує, скільки мілісекунд минуло.
Цикл **while** повторюється, поки **elapsed** менший за затримку цього стану. На кожному кроці кнопка перевіряється, потім пауза **10 мс**, і лічильник збільшується на **10**
Для стану **1** це **70** ітерацій (**700 мс**), для стану **4** лише **40** (**400 мс**). Такий підхід замінює один довгий **delay(...)**, під час якого кнопка була б «глуха».

```
void dualLights()
{
  showState(idx);

  idx += dir;
  if (idx > 3) idx = 0;
  if (idx < 0) idx = 3;
}
```

Одна «порція» роботи: показати стан, потім перейти до наступного. **idx += dir** додає або віднімає одиницю. Дві умови замикають послідовність у кільце: після **3** іде **0**, а перед **0** іде **3**

```
void setup()
{
  for (int k = 0; k < 4; k++)
    pinMode(led[k], OUTPUT);

  pinMode(S2, INPUT_PULLUP);
}

void loop()
{
  dualLights();
}
```

У **setup()** усі чотири світлодіодні піни стають виходами (знову циклом), а пін кнопки отримує **INPUT_PULLUP**. Кнопка підключається між пінами **2** і **GND**. **loop()** просто викликає **dualLights()** без кінця

Завдання 7:

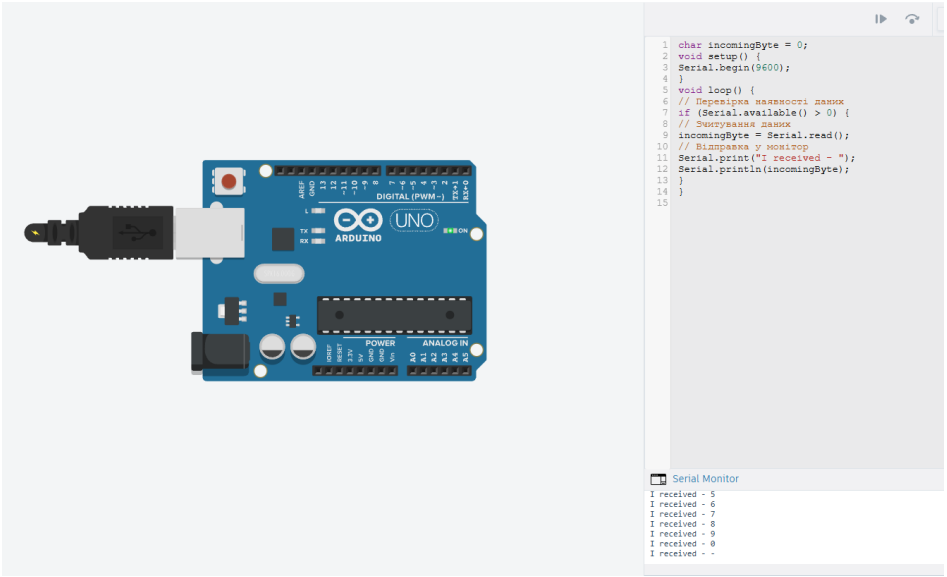


Рисунок 2.7 - перевірка роботи програми

Прогама: https://www.tinkercad.com/things/8e3xfikLxpt-lab27?sharecode=Q_l5KidpwHXD9D5vxJjCI1_1K-A28BtmGwpEITTwQsw

Як працює програма:

Програма приймає символи з комп'ютера (з монітора порту) і відповідає, що саме вона отримала.

char incomingByte = 0;
char це змінна на **1** байт, один символ. Вона зберігає останній прийнятий байт. Початкове значення **0** не має значення, бо воно одразу перезаписується.

```
void setup() {
  Serial.begin(9600);
}
```

Serial.begin(9600) запускає послідовний зв'язок зі швидкістю **9600 біт/с**. У вікні монітора порту має стояти така сама швидкість, інакше замість тексту з'являться випадкові символи. **pinMode** тут не потрібен: апаратний **UART** сам займає піни **0 (RX)** і **1 (TX)**, тому їх не можна використовувати для іншого, поки працює **Serial**

```
void loop() {
  if (Serial.available() > 0) {
    incomingByte = Serial.read();
    Serial.print("I received - ");
    Serial.println(incomingByte);
  }
}
```

Serial.available() повертає, скільки байтів уже прийшло й чекає в буфері (його розмір **64** байти). Якщо їх більше нуля, є що читати. Поки нічого не надіслали, **loop()** просто крутиться вхолосту.
Serial.read() забирає один байт із буфера й видаляє його звідти. Результат записується в **incomingByte**.
Serial.print("I received - ") виводить текст без переходу на новий рядок.
Serial.println(incomingByte) виводить прийнятий символ і переходить на новий рядок

Завдання 8:

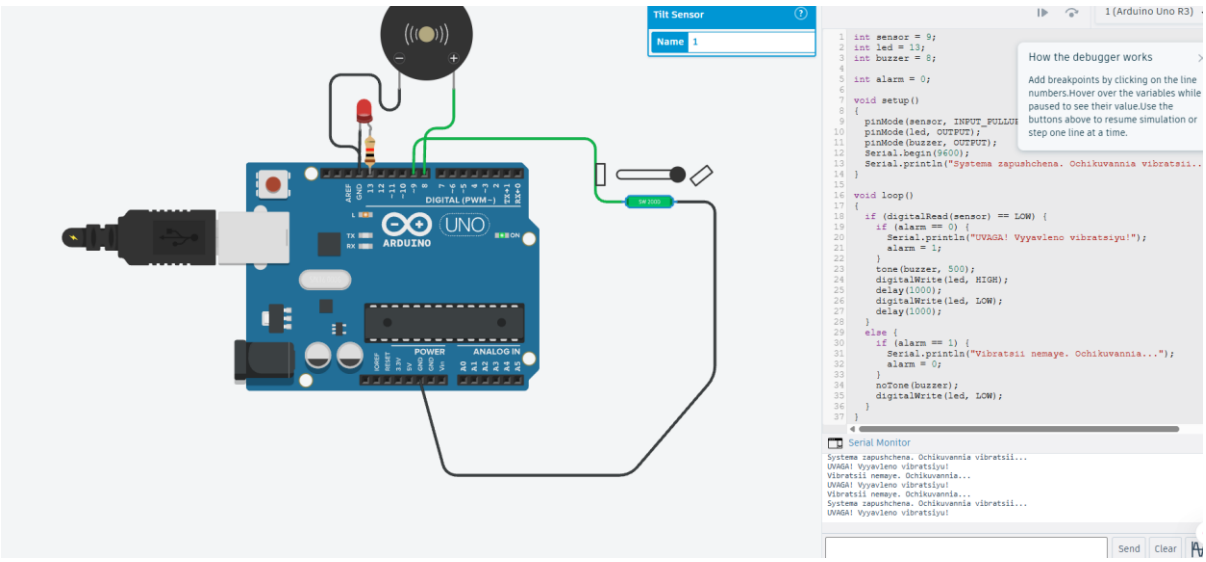


Рисунок 2.8- перевірка роботи програми

Програма: https://www.tinkercad.com/things/cirQVOeAMuD-lab28?sharecode=5lmZQL0XIcNVFYwRrzukF_rdb3GpEnbY9x6t90a2sLQ

Як працює програма:

Датчик вібрації підключений до піна **9**. Коли він спрацьовує, програма:
один раз пише повідомлення в послідовний монітор;
вмикає звук **500 Гц** на зумері;
блимає світлодіодом із частотою **0,5 Гц**.

Коли вібрація зникає, звук і світло вимикаються, і в монітор іде повідомлення про це

int sensor = 9;
int led = 13;
int buzzer = 8;
int alarm = 0;
Перші три змінні це номери пінів. **alarm** це прапорець стану: **0** означає «тривоги немає», **1** означає «тривога вже оголошена». Він потрібен, щоб повідомлення в монітор виводилося один раз на подію, а не тисячі разів на секунду (пояснення нижче)

setup()
pinMode(sensor, INPUT_PULLUP);
pinMode(led, OUTPUT);
pinMode(buzzer, OUTPUT);

Serial.begin(9600);
Serial.println("Systema zapushchena. Ochikuvannia vibratsii...");
INPUT_PULLUP вмикає внутрішній резистор на **+5 В**. Поки датчик розімкнений, пін **9** читається як **HIGH**. Коли датчик спрацьовує й замикає пін на **GND**, читається **LOW**. Тому логіка інвертована: **LOW** означає «вібрація є»
Serial.begin(9600) запускає послідовний зв'язок зі швидкістю **9600** бод. У вікні монітора порту має стояти така сама швидкість, інакше замість тексту буде сміття
Serial.println(...) надсилає рядок у монітор і переходить на новий рядок. Це стартове повідомлення, воно виводиться один раз.
Повідомлення написані латиницею (транслітом), бо послідовний монітор A**rduino IDE** та **Tinkercad** не завжди коректно показує кирилицю
loop(): гілка «вібрація є»

```
if (digitalRead(sensor) == LOW) {
    if (alarm == 0) {
      Serial.println("UVAGA! Vyyavleno vibratsiyu!");
      alarm = 1;
    }
    tone(buzzer, 500);
    digitalWrite(led, HIGH);
    delay(1000);
    digitalWrite(led, LOW);
    delay(1000);
}
```

digitalRead(sensor) == LOW перевіряє, чи датчик спрацював.
Вкладений **if (alarm == 0)** спрацьовує лише при першому виявленні. Виводиться повідомлення, і **alarm** стає **1**. Наступні проходи циклу цей блок пропускають, тож монітор не засмічується.
tone(buzzer, 500) подає на зумер меандр **500 Гц**, тобто звук потрібної частоти. Функція працює у фоні, тому звук триває далі, поки не буде **noTone**.
Світлодіод горить **1** с, потім гасне на **1** с. Період **T = 2 с**, частота **f = 1/T = 0,5 Гц**, як і вимагає завдання.
loop(): гілка «вібрації немає»

```
else {
    if (alarm == 1) {
      Serial.println("Vibratsii nemaye. Ochikuvannia...");
```

```
        alarm = 0;
    }
    noTone(buzzer);
    digitalWrite(led, LOW);
}
```

Якщо тривога щойно закінчилася (**alarm == 1**), виводиться повідомлення, і прапорець скидається в **0**. Знову ж, лише один раз.
noTone(buzzer) вимикає звук, а **digitalWrite(led, LOW)** гасить світлодіод.
У цій гілці немає **delay**, тому **loop()** крутиться дуже швидко, постійно перевіряючи датчик. Повторні виклики **noTone і digitalWrite** безпечні

Висновок: В ході лабораторної роботи ми ознайомились з принципами введення/виведення цифрових сигналів. Вивчити роботу дискретних електронних компонентів (світлодіод, пьезовипромінювач, кнопка тощо) та використання плати **Arduino**. Ознайомились з середовищем розробки **Arduino IDE**